

Frostline SOS — Development Documentation

[Link to the github repository](#) containing the code, and the gameplay video.

[Link to gameplay video](#)

Game Overview

Frostline SOS is a small 3D game made in Godot 4.

You walk on a snowy mountain, collect radio parts, and try to survive the cold.

Movement uses WASD/Arrows + Space to jump. The camera guides the movement direction.

Snow falls, fog limits distance, and a frost overlay slowly covers the screen to show how cold it gets.

HUD shows temperature and how many parts you collected.

At the end there is a screen with Replay and Exit.

Game Conditions

- **Win:** collect all **5** radio parts.
- **Temperature:** starts warmer and drops over time until it reaches **-40 °C**.
- **Lose:** if temperature reaches **-40 °C** before you collect all parts, the player “freezes” and the game ends.

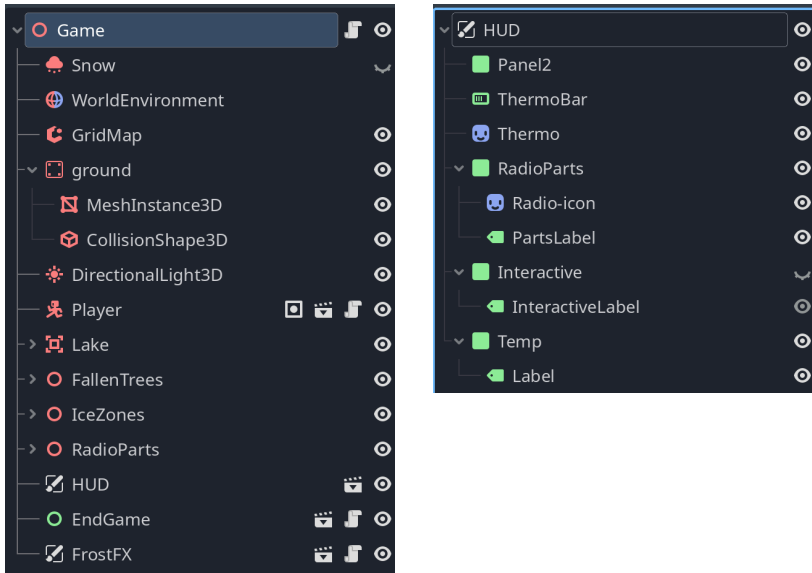
How it's implemented:

- The Game scene keeps an internal **timer**.
- Each frame, the game converts the timer progress into a **temperature value** (from about -5 °C down to -40 °C).
- The same progress also drives the **frost overlay** and slightly **reduces player speed** on normal ground (never as slow as ice).
- When time is up, the game checks: if all parts are collected → win, else → lose.

Scene Hierarchy

- **MainMenu.tscn** (*Control*)
 - Script: **main_menu.gd**
 - Start button loads the Game scene and captures the mouse.

- Replay does the same. Exit quits.
- **Game.tscn** (*Node3D*)



- Script: **game.gd** (controls timer, temperature, HUD, frost, win/lose)
- **HUD** (*CanvasLayer*)
 - **RadioParts/PartsLabel** (*Label*) → shows “collected / total”.
 - **Interactive** (*Panel*) → shows “[C] Pick up ...” when near a part.
 - **Temp/Label** (*Label*) → shows the current temperature in “-XX °C” format.
 - **ThermoBar** (*ProgressBar*) → visual temperature drop.
- **EndGame** (*Control*) → win/lose text, replay/exit flow.
- **FrostFX** (*CanvasLayer*) → full-screen frost effect (shader).
- **Player.tscn** (*CharacterBody3D*)
 - Script: **player.gd**
 - **Camera_Controller** (*Node3D*) → rotates with mouse; move is camera-relative.
 - **Barbarian/AnimationPlayer** → movement, jump, pickup, win/lose animations.
- **RadioPart.tscn** (*Area3D*)
 - Script: **radioPart.gd**
 - **CollisionShape3D** (*Sphere*) → proximity trigger.
 - **Highlight/Ring** (*MeshInstance3D*) → spins/pulses when player is near.
- **IceZone.tscn** (*Area3D*)
 - Script: **ice_zone.gd**

- A volume that makes the floor slippery while the player is inside.

Player Abilities / Movement and Collecting

Movement

- The player is a CharacterBody3D with gravity and **jump** on Space.
- Movement is **camera-relative**:
 - W moves toward where the camera looks.
 - A/D and S work the same way, based on the camera orientation.
- The character mesh rotates to face the move direction.
- On the ground, base speed is slightly reduced over time by the **cold factor** from the Game script. This makes the pressure visible but the player still stays in control.

Collecting Radio Parts



Each radio part is an Area3D with a spherical trigger (CollisionShape3D→SphereShape3D).

When the player enters the sphere, the item becomes visually emphasized (spinning/pulsing/ring).

Collection happens instantly on overlap—no button press required. The system:

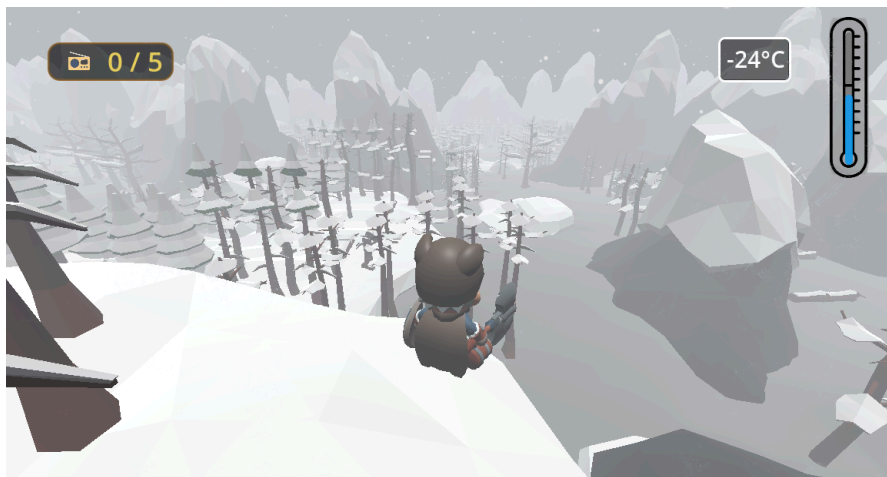
- Increments the collected-parts counter.
- Triggers the player's short pickup animation (if available).
- Hides any active prompt.
- Removes the part from the scene (queue_free()).

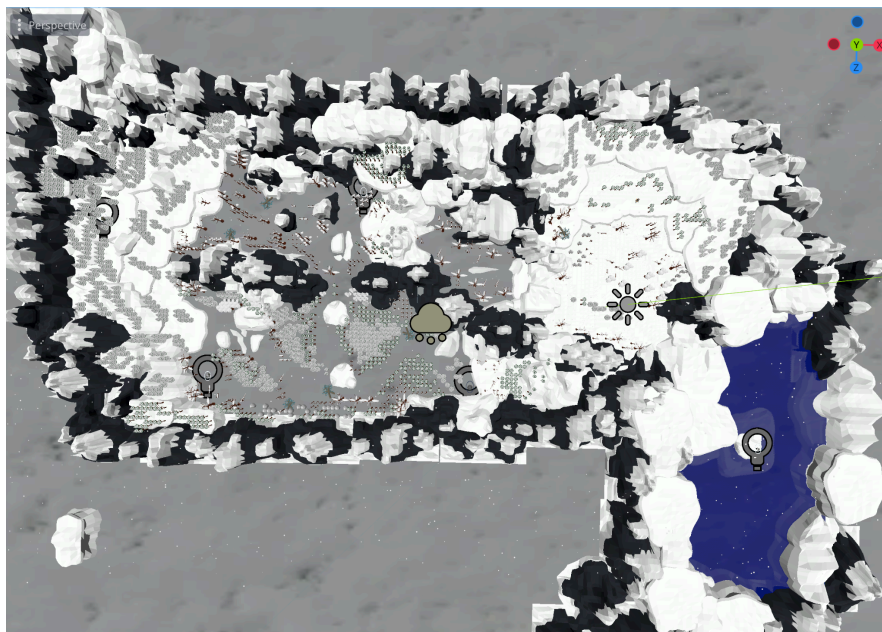
Animations

- Idle/Walk switch based on horizontal speed.
- Jump plays on lift-off.
- Pickup plays a short one-shot and briefly ignores gait changes so the pickup looks clean.
- Win plays a “cheer”; lose plays a “death” one-shot.

Map/ World

The world is made of tough mountain terrain, sharp mountain peaks, snowy hills. Equipped with fallen trees snowy trees and fallen logs on the ground.





Ice Zones (Slippery Areas)

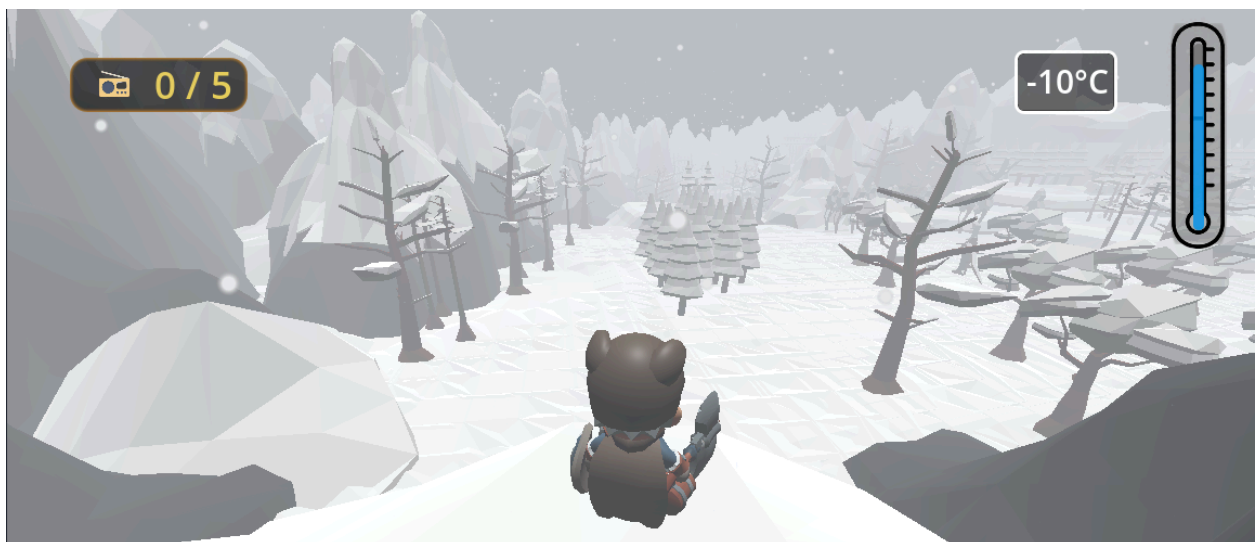


Goal: make ice feel slippery without breaking control. Only appears when the player is at a close distance to it.

How it works:

- **Ice zones** are **Area3D** volumes placed over icy ground.
- When the player enters:
 - The zone **lowers max speed** (e.g., to 65%).
 - **Acceleration** becomes slow (takes time to get going).
 - **Deceleration** becomes very slow (long slide to stop).
- While on ice, horizontal velocity moves **gradually** toward a target using smoothing, not instant snaps.
- If the player lets go of input, the **facing** follows the **slide direction**, which matches what you see.
- When the player leaves the zone, normal ground settings return immediately.

Snowfall Over the Map



Goal: lightweight, always-visible snow.

How it works:

- A **GPUParticles3D** node emits simple quads across a **large box** above the level.
- Flakes fall down with small turbulence so they don't look identical.
- **Scale** and **lifetime** curves make flakes appear, grow a bit, then fade out softly.
- A **color ramp** fades to transparent at the end.
- “**Generate AABB**” avoids particles popping out when the camera angle changes.

Fog on the Map

Goal: give depth and reduce far-distance clutter.

How it works:

- A **WorldEnvironment** enables **fog** with light gray color and moderate density.
- This makes far trees and rocks fade into the background, while nearby objects stay clear.
- A bit of sky scatter helps the image look natural with the snow.

Frost Overlay (Screen Freezing)



Goal: show the cold getting worse, right on the screen.

How it works:

- A **CanvasLayer** holds a full-screen **ColorRect** with a **shader**.
- The Game script sends a **freeze value** from **0 to 1** as time passes.
- The shader uses that value to **grow ice** from the edges and soften what's behind it.
- A time uniform adds a slow movement so it doesn't look static.
- Because it's on a CanvasLayer, it sits cleanly over 3D and under the HUD.

Temperature on the HUD



- The HUD **Label** shows the current value (e.g., **-23 °C**).
- The **ThermoBar** drops along with it.
- Both are updated every frame by the Game script based on the **timer** → **temperature mapping**.
- The same value also drives the frost overlay and the small ground speed reduction, so what you see, what you feel, and what the UI shows all match.

Endgame Screen (Replay / Exit)

- When the round ends, a full-screen **EndGame** layer appears.
- **Texts:**
 - Win: „Активиран сигнал – Спасен си!“
 - Lose: „Се е смрзнато! Обиди се повторно...“
 - A bold state label: **YOU WIN** or **GAME OVER**.
- **Replay:** reloads the Game scene (fresh timer, fresh parts).
- **Exit:** quits the application.
- Start and Replay use the same path so each attempt starts clean.

